Hands-On Activity 10 | Explaining Visually to Influence with Data

Name : Mendoza, Ryo Shin P.
Course Code and Title : CPE031 - Visulaization and Data Analysis
Date Submitted : October 10, 2026
Instructor : Engr. Jimlord Quejado

1. Objectives:

This activity aims to help students demonstrate how to influence and communicate effectively with data by identifying audience needs, crafting a clear message, and telling a story visually through charts.

2. Intended Learning Outcomes (ILOs):

By the end of this activity, students should be able to:

1. Analyze the audience to understand their level of data knowledge and needs.
2. Evaluate how well a message and visualization communicate insights.
3. Create a clear and meaningful data story using Python visualization tools.

3. Discussions:

Data storytelling goes beyond showing charts it's about influencing decisions and sparking understanding.

Know Your Audience: Decision-makers may prefer clear summaries, while analysts like detailed data.

Know Your Message: Focus on the why it matters part of the story.

Tell a Story: Use preattentive attributes color, size, and position to guide attention.

Structure Your Story: Like Aristotle's rhetorical triangle, combine logic (logos), emotion (pathos), and credibility (ethos).

4. Procedures:

Step 1. Import Libraries and Create Sample Data

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Sample sales dataset
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [200, 250, 180, 300, 280, 350],
    "Marketing_Spend": [50, 60, 45, 70, 65, 80]
}
df = pd.DataFrame(data)
df
```

	Month	Sales	Marketing_Spend
0	Jan	200	50
1	Feb	250	60
2	Mar	180	45
3	Apr	300	70
4	May	280	65
5	Jun	350	80

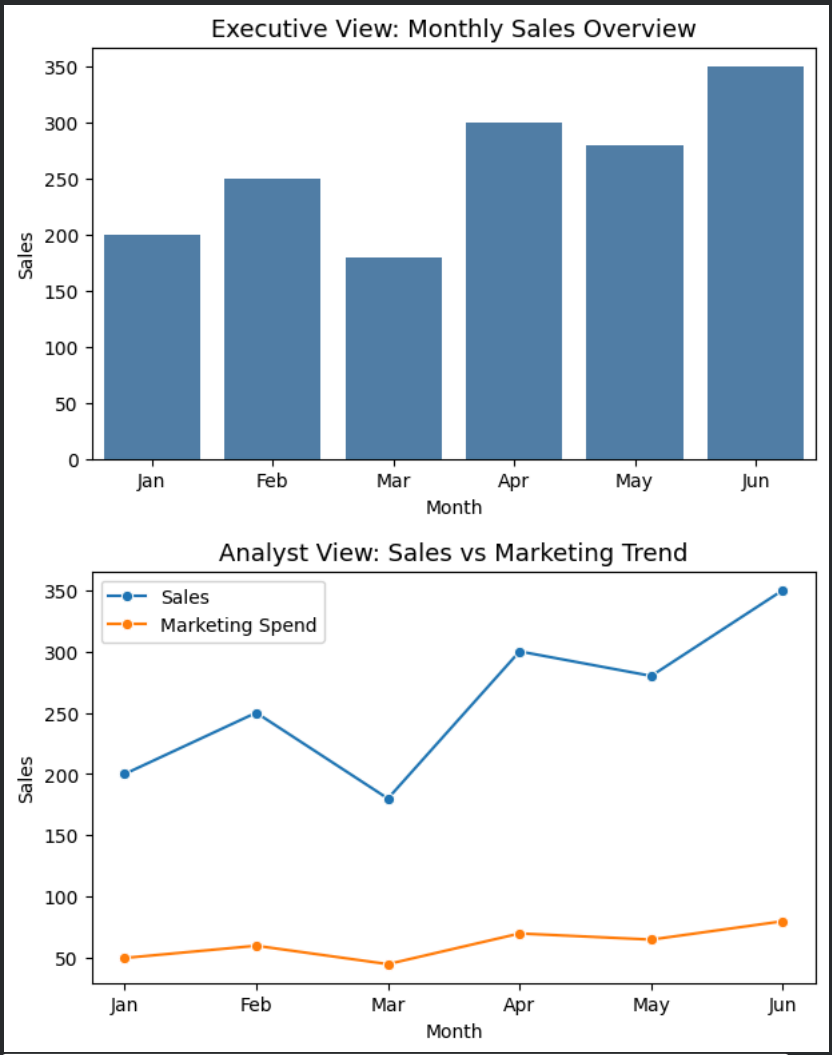
Next steps: Generate code with df | New interactive sheet

Step 2. Know Your Audience — Compare Chart Styles

```
# Executive audience: prefer clarity (simple bar)
plt.figure(figsize=(7,4))
sns.barplot(x="Month", y="Sales", data=df, color="steelblue")
```

```
plt.title("Executive View: Monthly Sales Overview", fontsize=13)
plt.show()

# Analyst audience: prefer detailed trends (line chart)
plt.figure(figsize=(7,4))
sns.lineplot(x="Month", y="Sales", data=df, marker="o", label="Sales")
sns.lineplot(x="Month", y="Marketing_Spend", data=df, marker="o", label="Marketing Spend")
plt.title("Analyst View: Sales vs Marketing Trend", fontsize=13)
plt.legend()
plt.show()
```

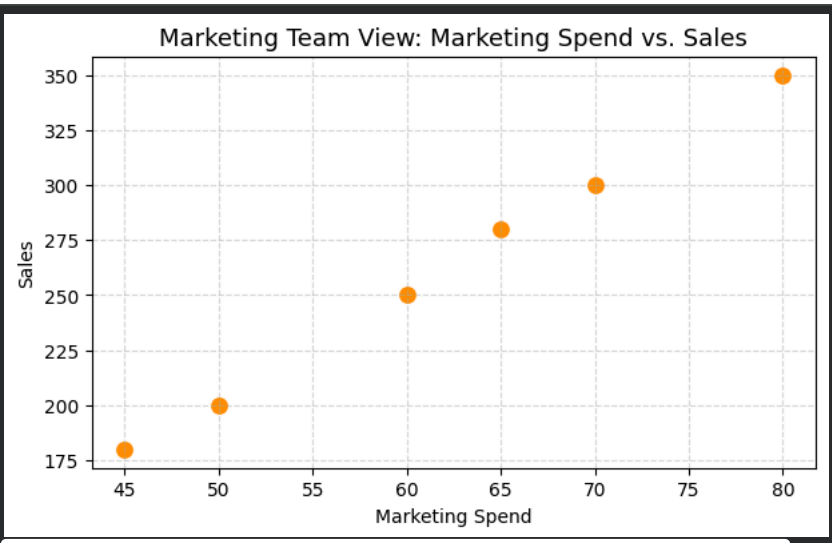


Task 1: Create one additional visualization suited for a different audience (e.g., a customer, investor, or teammate).

```
# Marketing Team View: Marketing Spend vs. Sales
plt.figure(figsize=(7, 4))

sns.scatterplot(
    x="Marketing_Spend",
    y="Sales",
    data=df,
    color="darkorange",
    s=100
)

plt.title("Marketing Team View: Marketing Spend vs. Sales", fontsize=13)
plt.xlabel("Marketing Spend")
plt.ylabel("Sales")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



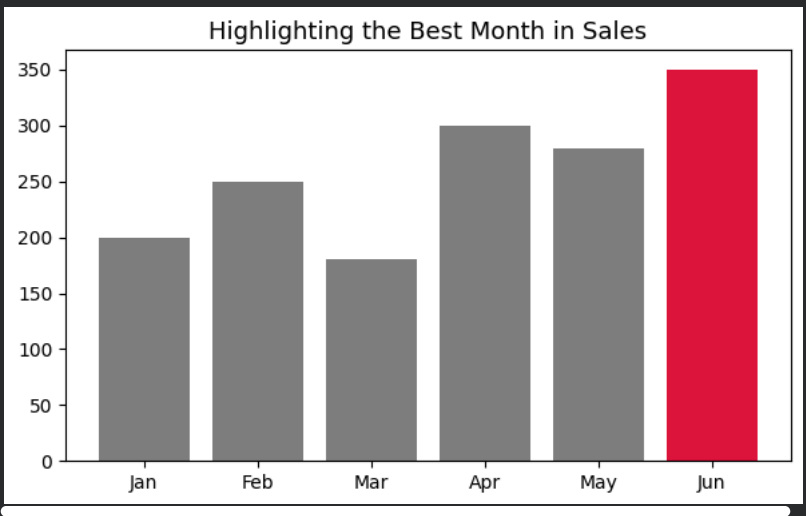
Analysis

The scatter plot shows the relationship between marketing spending and sales for six months. The points generally move upward as marketing spending increases, suggesting a positive relationship between the two variables. This visualization helps the marketing team examine how spending relates to sales and supports planning for future marketing activities. However, it does not prove that increased marketing spending directly causes higher sales.

Step 3. Know Your Message — Highlight What Matters

```
# Highlight the highest sales month (preattentive color cue)
highlight = df["Sales"].idxmax()

colors = ["gray" if i != highlight else "crimson" for i in range(len(df))]
plt.figure(figsize=(7,4))
plt.bar(df["Month"], df["Sales"], color=colors)
plt.title("Highlighting the Best Month in Sales", fontsize=13)
plt.show()
```



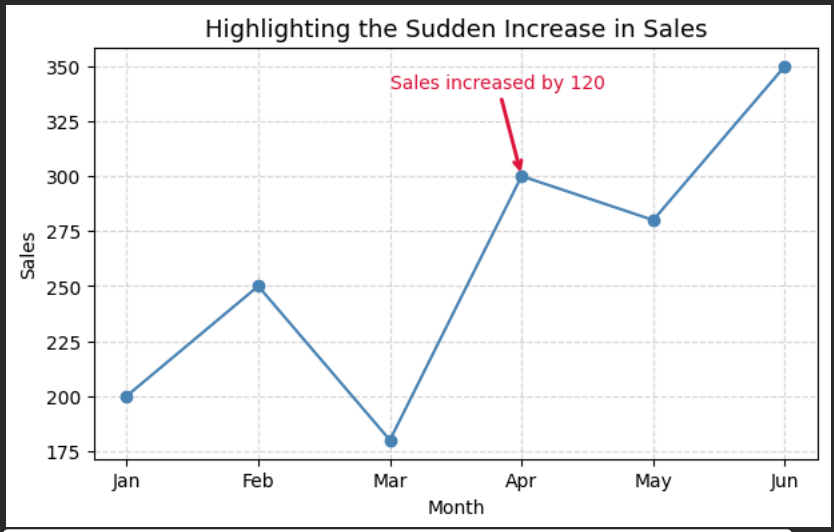
Task 2: Use color or annotations to emphasize a key insight (e.g., lowest point, sudden increase).

```
# Highlight the sudden increase in sales from March to April
plt.figure(figsize=(7, 4))

plt.plot(df["Month"], df["Sales"], marker="o", color="steelblue")

plt.annotate(
    "Sales increased by 120",
    xy=("Apr", 300),
    xytext=("Mar", 340),
    color="crimson",
    fontsize=10,
    arrowprops=dict(arrowstyle="->", color="crimson", lw=2)
)
```

```
plt.title("Highlighting the Sudden Increase in Sales", fontsize=13)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



Analysis

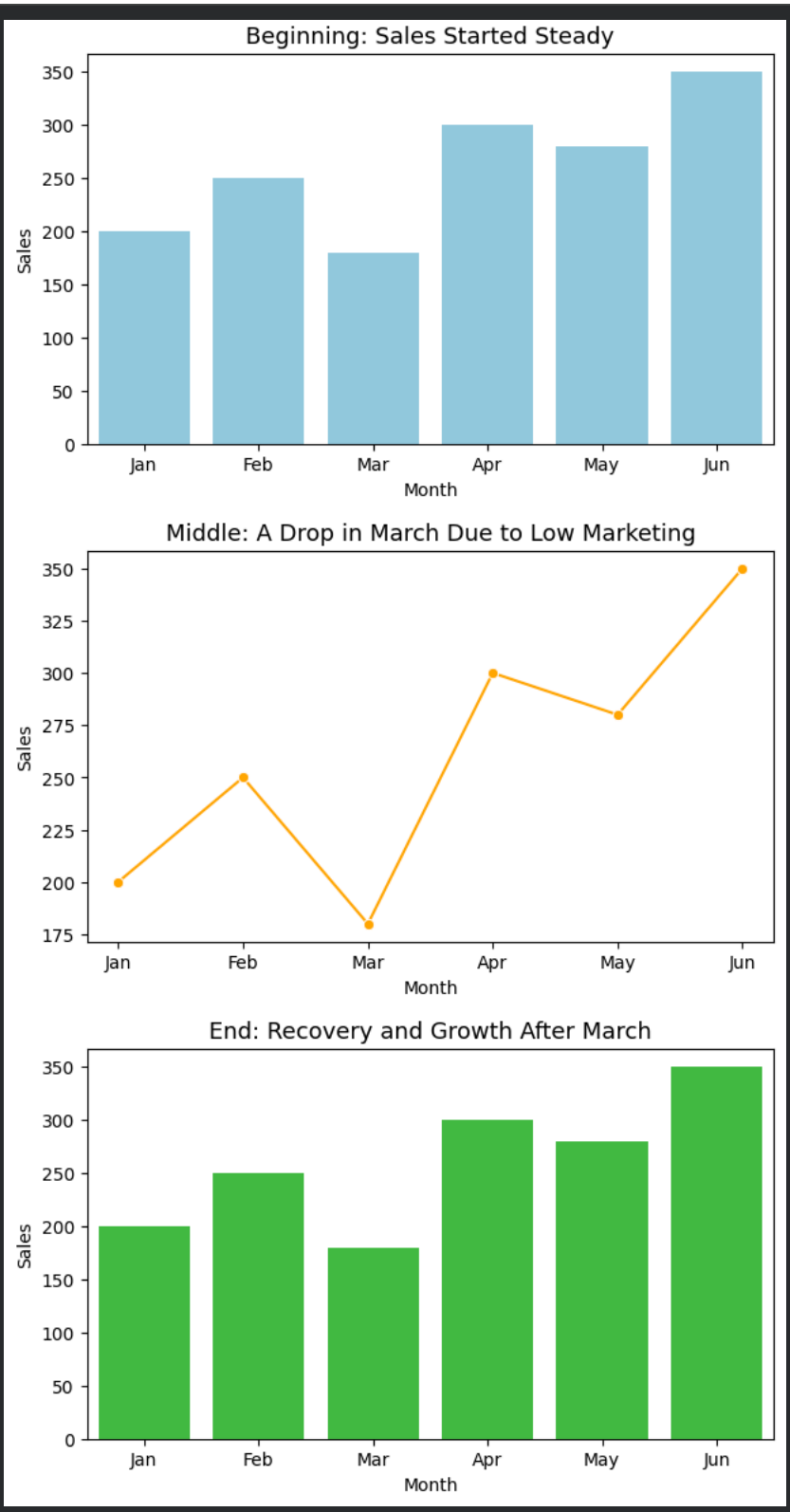
The line chart highlights the increase in sales from March (180) to April (300), representing an increase of 120. The crimson annotation and arrow direct the audience's attention to this change, making the key insight easier to understand.

Step 4. Storytelling with Charts — Beginning, Middle, End

```
# Beginning: Context
plt.figure(figsize=(7,4))
sns.barplot(x="Month", y="Sales", data=df, color="skyblue")
plt.title("Beginning: Sales Started Steady", fontsize=13)
plt.show()

# Middle: Change or Challenge
plt.figure(figsize=(7,4))
sns.lineplot(x="Month", y="Sales", data=df, color="orange", marker="o")
plt.title("Middle: A Drop in March Due to Low Marketing", fontsize=13)
plt.show()

# End: Resolution
plt.figure(figsize=(7,4))
sns.barplot(x="Month", y="Sales", data=df, color="limegreen")
plt.title("End: Recovery and Growth After March", fontsize=13)
plt.show()
```



Task 3: Write one or two lines describing your data story from beginning → middle → end.

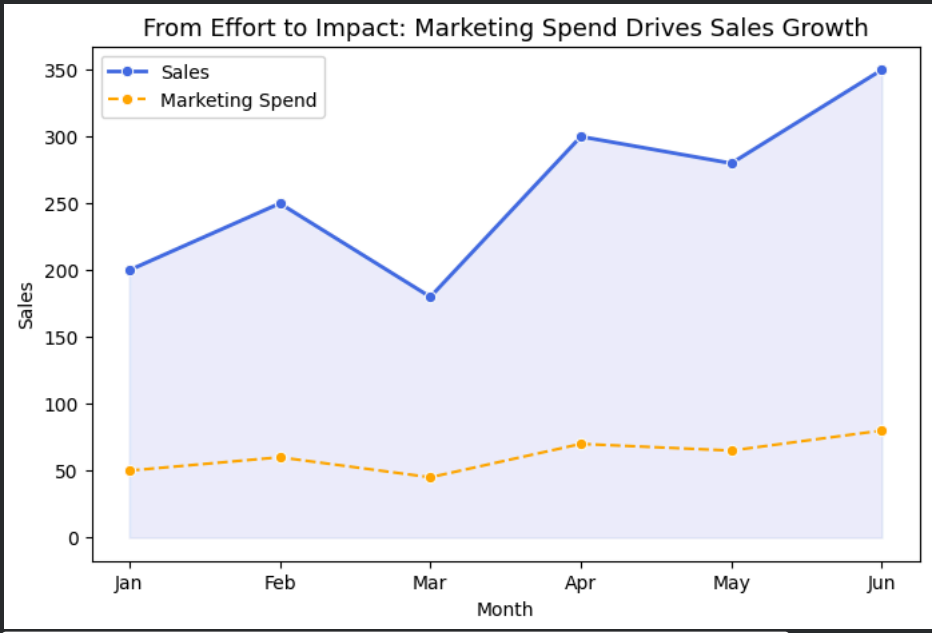
At the beginning, sales were relatively steady, but they dropped in March before increasing again in April. By the end, sales continued to grow and reached their highest point in June, showing an overall positive trend.

Step 5. Bringing It All Together — Presentation Design

```
# Simple combined story chart
plt.figure(figsize=(8,5))
sns.lineplot(x="Month", y="Sales", data=df, marker="o", linewidth=2, color="royalblue", label="Sales")
sns.lineplot(x="Month", y="Marketing_Spend", data=df, marker="o", linestyle="--", color="orange", label="Marketing Spend")

plt.fill_between(df["Month"], df["Sales"], alpha=0.1, color="royalblue")
plt.title("From Effort to Impact: Marketing Spend Drives Sales Growth", fontsize=13)
```

```
plt.legend()
plt.show()
```



---

5. Supplementary Activity:

1. Use a small dataset available online (kaggle, google dataset, etc.).

I used this dataset from Kaggle: https://www.kaggle.com/code/mdismielhossenabir/pizza-sales-dataset/

2. Visualize it using bar chart

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

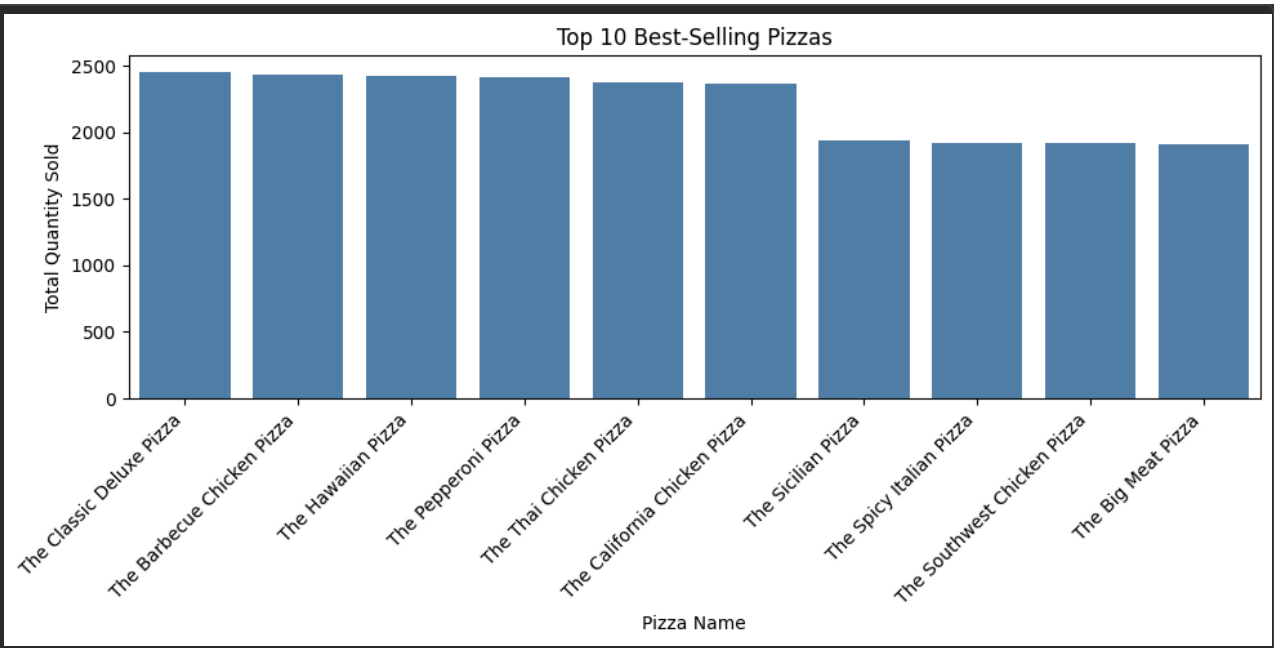
# Load the Kaggle dataset
df = pd.read_csv("pizza_sales.csv")

# Get total quantity sold for each pizza
pizza_sales = df.groupby("pizza_name")["quantity"].sum()

# Select the top 10 best-selling pizzas
top_10 = pizza_sales.sort_values(ascending=False).head(10)

# Create the bar chart
plt.figure(figsize=(10, 5))
sns.barplot(x=top_10.index, y=top_10.values, color="steelblue")

plt.title("Top 10 Best-Selling Pizzas")
plt.xlabel("Pizza Name")
plt.ylabel("Total Quantity Sold")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



3. Highlight one important data point (e.g., the highest or lowest value) using a distinct color.

```
# Calculate total quantity sold per pizza
pizza_sales = df.groupby("pizza_name")["quantity"].sum()

# Get the top 10 best-selling pizzas
top_10 = pizza_sales.sort_values(ascending=False).head(10)

# Identify the highest value
highest = top_10.idxmax()

# Highlight the best-selling pizza
colors = [
    "crimson" if pizza == highest else "steelblue"
    for pizza in top_10.index
]

# Create the bar chart
plt.figure(figsize=(10, 5))
plt.bar(top_10.index, top_10.values, color=colors)

plt.title("Top 10 Best-Selling Pizzas")
plt.xlabel("Pizza Name")
plt.ylabel("Total Quantity Sold")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



Analysis: The chart highlights the best-selling pizza in crimson color, while the other pizzas are shown in steel blue. This makes it easier for the audience to identify the pizza with the highest total quantity sold.

4. Add a title and annotation that tell the story behind the data.

```
# Calculate total quantity sold per pizza
pizza_sales = df.groupby("pizza_name")["quantity"].sum()

# Select the top 10 best-selling pizzas
top_10 = pizza_sales.sort_values(ascending=False).head(10)

# Identify the best-selling pizza
highest = top_10.idxmax()
highest_sales = top_10[highest]

# Highlight the highest value
colors = [
    "crimson" if pizza == highest else "steelblue"
    for pizza in top_10.index
]

# Create the bar chart
plt.figure(figsize=(10, 5))
plt.bar(top_10.index, top_10.values, color=colors)

# Add title and annotation
plt.title("Top 10 Best-Selling Pizzas: One Customer Favorite Stands Out")
plt.xlabel("Pizza Name")
plt.ylabel("Total Quantity Sold")
plt.xticks(rotation=45, ha="right")

plt.annotate(
    f"Best seller: {highest}\n{highest_sales} pizzas sold",
    xy=(highest, highest_sales),
    xytext=(0, 20),
    textcoords="offset points",
    ha="center",
    color="crimson",
    fontweight="bold",
    arrowprops=dict(arrowstyle="->", color="crimson")
)

plt.tight_layout()
plt.show()
```

